Justificativa dos Bancos de Dados

Conexão Solidária · Hackathon 11NETT · Entregável 2b · Setembro de 2026

1. Resumo da decisão

A plataforma é composta por dois microsserviços, cada um dono do seu próprio banco de dados (*database per service*). Foram escolhidos **dois bancos de naturezas diferentes**, porque os dois serviços têm necessidades de dados diferentes:

Serviço	Banco	Responsabilidade sobre os dados
CampanhasAPI	**PostgreSQL 17**	Usuários, campanhas e intenções de doação — dados transacionais, relacionais, com integridade referencial.
DoacoesWorker	**MongoDB 8**	Resumo por campanha com o *valor total arrecadado* (modelo de leitura do painel público) e o ledger de idempotência das doações processadas.

> O **valor total arrecadado existe apenas no MongoDB do Worker**. A entidade `Campanha` da API não possui esse campo — o que torna impossível, mesmo por engano, a API atualizá-lo diretamente, como exige o requisito técnico de mensageria.

2. Por que PostgreSQL para a CampanhasAPI

Os dados da API são o que o enunciado descreve como regras de integridade:

- **E-mail único no banco** (RF-3): índice único em `usuarios.Email`. O banco garante, não só o código.
- **Doação pertence a uma campanha e a um doador**: chaves estrangeiras `doacoes → campanhas` e `doacoes → usuarios`, com `ON DELETE RESTRICT`.
- **Dinheiro com precisão exata**: `MetaFinanceira` e `Valor` como `numeric(18,2)`. Ponto flutuante é inaceitável para valores financeiros.
- **Transação ACID** no fluxo "gravar a doação como pendente → publicar o evento → marcar como publicada".

Além da adequação ao modelo, pesaram fatores operacionais:

- **Entity Framework Core com Npgsql**: migrations versionadas no repositório, aplicadas automaticamente no startup — o mesmo fluxo já praticado pelo time com SQL Server em fases anteriores.
- **Leveza no cluster local**: a imagem `postgres:17` tem ~150 MB e roda com dezenas de MB de RAM. A alternativa natural, SQL Server, tem imagem de ~1,5 GB e exige 2 GB de RAM para

iniciar — num Kubernetes local que também hospeda RabbitMQ, MongoDB, Prometheus e Grafana, isso derruba pods.

3. Por que MongoDB para o DoacoesWorker

O Worker não mantém entidades relacionais; mantém **projeções derivadas de eventos**:

- `campanhas_resumo` — um documento por campanha, já no formato que o painel de transparência devolve (título, meta, status, total arrecadado, quantidade de doações). O endpoint público lê um documento pronto, sem *join* e sem tocar no banco transacional da API.
- `doacoes_processadas` — ledger *append-only*, chave = `IdDoacao`. É o que garante que uma doação reentregue pelo broker não seja somada duas vezes.

Três recursos do MongoDB são usados diretamente pelo desenho:

Recurso	Uso
`$inc` atômico	Somar o valor da doação ao total sem ler-modificar-gravar — seguro com várias réplicas do Worker.
`upsert` + `$setOnInsert`	Os eventos de campanha e de doação chegam por filas diferentes, sem ordem garantida. Qualquer um dos dois cria o documento; o outro completa, sem zerar o total.
Chave primária `_id` única	Inserção no ledger falha por chave duplicada na reentrega — a idempotência é decidida pelo banco, não por uma consulta seguida de inserção (que teria condição de corrida).

Valores monetários são armazenados como `Decimal128`, nunca como `double`.

4. Por que dois bancos, e não um

Um único banco compartilhado seria mais simples de operar, mas acoplaria os dois serviços pelo *schema*: o Worker escreveria na tabela da API, e qualquer mudança de modelo em um exigiria coordenação com o outro. Com bancos separados, o único contrato entre eles é o **evento** (`DoacaoRecebidaEvent`, `CampanhaAtualizadaEvent`). Cada serviço pode evoluir, escalar e falhar de forma independente — que é o argumento central da arquitetura de microsserviços.

O desenho resultante é CQRS: a API é o lado de escrita (comandos, transações), o Worker é o lado de leitura (projeções), e o RabbitMQ carrega a projeção de um para o outro. A consistência é eventual — segundos — exatamente o comportamento que o enunciado descreve ao dizer que o valor arrecadado é atualizado por um segundo serviço.

5. Alternativas consideradas e descartadas

Alternativa	Por que não
SQL Server no lugar do PostgreSQL	Já conhecido pelo time, mas 10× mais pesado em imagem e memória. Inviável no cluster local ao lado dos demais componentes. Para o EF Core a diferença é só o provider.
Um único PostgreSQL com duas *databases*	Atende *database per service*, mas não traz benefício de modelo para o Worker — que precisa de `$inc`, upsert e documentos, não de tabelas.
A API lendo o MongoDB do Worker para servir o painel	Menos código, mas cria um segundo canal de acoplamento (o schema do Mongo) além do evento. O painel foi colocado no Worker, que é o dono do dado.
Redis como segundo banco	Serviria como cache do painel, mas não como fonte de verdade do total arrecadado nem como ledger durável.

Este documento consolida as decisões registradas no repositório em `docs/decisoes/001-bancos-de-dados.md` e `003-painel-de-transparencia.md`. Repositório: github.com/pablosdlima/ConexaoSolidaria